

Note 13: base of type theory and Curry-Howard isomorphism

Author: JiaHao Liang

Date: 2026.7.26

13.1 Summary of Progress

1. Establish complete QECC correction conditions and error discretization theorem with rigorous proofs.
2. Systematically derive diamond norm definitions, core properties and composite noise bound for channel error quantification.
3. State and fully prove the concatenated quantum fault-tolerance threshold theorem based on quadratic error recursion.

13.2 Type theory

In dependent type theory, there is no set-theoretic style of one-shot extensional definition. We cannot write down an isolated definition of "what a type is", because a "type" constitutes a syntactic category, whose well-formedness can only be characterized within the deductive system of contexts.

Notational Convention.

- Uppercase Greek letters Γ, Δ : contexts;
- Turnstile $\vdash$: derivable judgement in deductive system;
- Symbol $\equiv$: definitional equality induced by computation rules;
- All terms: identified up to α -equivalence (variable renaming).

Preliminary Recursive Stratification of Syntactic Objects

Definition 1 We first define the three fundamental layers of syntactic entities in dependent type theory. These layers classify all valid symbolic objects of the system.

- Context Γ, Δ : a finite ordered sequence of variable declarations $x_1 : A_1, \dots, x_n : A_n$, where each A_i is well-formed under the preceding prefix context.
- Type expression: a syntactic construct that can appear on the right-hand side of a type judgement.
- Term expression: a syntactic construct that can appear on the right-hand side of a typing judgement.

Remark 2 The above is static syntactic classification. The judgements below define deductive well-formedness and equivalence.

Definition 3 The deductive power of the system is entirely governed by four primitive judgements, which determine well-formedness, inhabitation and definitional equality:

1. $\vdash \Gamma \text{ ctx}$: Γ is a well-formed context;
2. $\Gamma \vdash A \text{ Type}$: A is a well-formed type under Γ ;
3. $\Gamma \vdash a : A$: term a inhabits type A under Γ ;
4. $\Gamma \vdash a \equiv b : A$: definitional equality, i.e., syntactic reduction equivalence between terms a, b .

Remark 4 Unlike ZFC set theory, which adopts $\in$ as a primitive first-order predicate, type theory takes inhabitation $a : A$ as a primitive syntactic judgement rather than a propositional formula. This yields the fundamental structural difference between type-theoretic formalization and set-theoretic formalization.

Context Formation Rules

To formalize valid contexts, we introduce two recursive inference rules.

Postulate 1 . Denote $\text{dom}(\Gamma)$ as the set of variable names declared inside Γ .

1. **Ctx-Emp**: The empty list is always a valid context

$$\vdash \emptyset \text{ ctx}$$

2. **Ctx-Ext**: We may extend a context with a fresh variable declaration

$$\frac{\vdash \Gamma \text{ ctx} \quad \Gamma \vdash A \text{ Type} \quad x \notin \text{dom}(\Gamma)}{\vdash \Gamma, x : A \text{ ctx}}$$

Well-Formedness of Types and Terms

All legitimate types and terms are defined derivatively, rather than via static set comprehension.

Definition 5 . Fix a valid context Γ . An expression A is a Γ -well-formed type if $\Gamma \vdash A \text{ Type}$ can be deduced through finite formation rules. There is no universal collection of all types, which syntactically blocks Russell-style paradoxes.

Definition 6 . Given a well-formed Γ and a Γ -type A , an expression a is a well-typed term of A when $\Gamma \vdash a : A$ is provable via introduction and elimination rules.

The judgement $\Gamma \vdash a \equiv b : A$ encodes definitional equality: two terms are identified whenever repeated computation reduction transforms them into identical syntax. This equivalence is native to the deductive system, not a semantic interpretation over external models.

Unified FIEC Rule Template

Every type constructor follows an identical four-rule schema, which unifies all type constructions and distinguishes type theory from the scattered axiom schemas of ZFC:

1. **Formation (F)**: States the preconditions for parameters and asserts the expression is a valid type;
2. **Introduction (I)**: Provides constructors to build elements of this type;
3. **Elimination (E)**: Supplies recursion/case analysis operators to destructure elements;
4. **Computation (C)**: Fixes reduction equalities when eliminators act on canonical constructors.

Definition 7 (General Recursor rec_T) The recursor rec_T is the standard elimination operator for an inductive type T as specified by the FIEC E-rule, for case analysis and structural decomposition.

Syntax:

$$\text{rec}_T(C, \text{branches}, t)$$

1. C : Uniform target output type for all branches.
2. branches : A list of expressions, one per constructor of T , defining results for each construction case.
3. t : Term of type T to be deconstructed.

Computation Rule: If t is a canonical constructor term $\text{Constr}_i(\dots)$, then

$$\text{rec}_T(C, \text{branches}, \text{Constr}_i(\dots)) \equiv \text{the } i\text{-th branch}$$

This reduction implements the FIEC C-rule.

Primitive Atomic Types

We present the three base atomic types with complete FIEC rules.

Example 8 Empty Type 0

Corresponds to logical absurdity; no terms inhabit this type.

- F: $\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \mathbf{0} \text{ Type}}$
- I: No introduction rules
- E (Ex Falso): $\frac{\Gamma \vdash a : \mathbf{0} \quad \Gamma \vdash C \text{ Type}}{\Gamma \vdash \text{rec}_0(C, a) : C}$
- C: No computation rules

Example 9 Unit Type 1

Singleton type encoding tautological propositions.

- F: $\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \mathbf{1} \text{ Type}}$
- I: $\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \star : \mathbf{1}}$
- E: $\frac{\Gamma \vdash C \text{ Type} \quad \Gamma, y : \mathbf{1} \vdash c : C \quad \Gamma \vdash a : \mathbf{1}}{\Gamma \vdash \text{rec}_1(C, c, a) : C}$
- C: $\Gamma \vdash \text{rec}_1(C, c, \star) \equiv c[\star/y] : C$

Example 10 Boolean Type $\mathbb{B}$

Two-valued inductive type for classical truth values.

- F: $\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \mathbb{B} \text{ Type}}$
- I: $\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \text{true} : \mathbb{B}}, \quad \frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \text{false} : \mathbb{B}}$
- E: Case branching operator $\text{if}_{\mathbb{B}}$

$$\frac{\Gamma \vdash C \text{ Type} \quad \Gamma \vdash c_t : C \quad \Gamma \vdash c_f : C \quad \Gamma \vdash b : \mathbb{B}}{\Gamma \vdash \text{if}_{\mathbb{B}}(C, c_t, c_f, b) : C}$$

- C: Two reduction rules

$$\Gamma \vdash \text{if}_{\mathbb{B}}(C, c_t, c_f, \text{true}) \equiv c_t : C, \quad \Gamma \vdash \text{if}_{\mathbb{B}}(C, c_t, c_f, \text{false}) \equiv c_f : C$$

Symbol Glossary

1. Γ : Typing context, a finite list of variable-type bindings $x_1 : A_1, \dots, x_n : A_n$.
2. $\Gamma \vdash t : T$: Typing judgment; under context Γ , term t has type T .

3. $\Gamma \vdash T \text{ Type}$: T is a well-formed valid type under Γ .
4. $t[a/x]$: Capture-avoiding substitution; replace all free occurrences of variable x inside term t with term a . Bound variables named x (bound by λ) remain unchanged.
5. $\lambda x. t$: Lambda abstraction, constructor for function types.
6. $f(a)$: Term application, apply function f to argument a .
7. $\equiv$: Definitional equality, reducible via computation rules.
8. inl, inr : Introduction constructors for disjoint sum types.

Non-dependent composite types

Suppose $\Gamma \vdash A \text{ Type}$, $\Gamma \vdash B \text{ Type}$.

1. **Product** $A \times B$: Ordered pairs (a, b)

$$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma \vdash B \text{ Type}}{\Gamma \vdash A \times B \text{ Type}}$$

- Introduction rule: Given $\Gamma \vdash a : A, \Gamma \vdash b : B$, construct pair $\Gamma \vdash (a, b) : A \times B$.
- Elimination: Recursor $\text{rec}_\times(C, g, p)$; reduction rule $\text{rec}_\times(C, g, (a, b)) \equiv g(a, b)$.

2. **Sum** $A + B$: Disjoint union via inl, inr constructors

$$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma \vdash B \text{ Type}}{\Gamma \vdash A + B \text{ Type}}$$

- Introduction rules:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}(a) : A + B}, \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}(b) : A + B}$$

- Elimination: Recursor $\text{rec}_+(C, f_1, f_2, s)$; reduction:

$$\text{rec}_+(C, f_1, f_2, \text{inl}(a)) \equiv f_1(a), \quad \text{rec}_+(C, f_1, f_2, \text{inr}(b)) \equiv f_2(b)$$

3. **Function** $A \rightarrow B$: λ -abstractions as elements

$$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma \vdash B \text{ Type}}{\Gamma \vdash A \rightarrow B \text{ Type}}$$

- Introduction rule: $\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x. t : A \rightarrow B}$
- Elimination rule: $\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B}$
- Computation rule (β -reduction): $(\lambda x. t)(a) \equiv t[a/x]$

Dependent composite types

Assume $\Gamma, x : A \vdash B(x) \text{ Type}$.

1. **Dependent product** $\prod_{x:A} B(x)$: Formal universal quantification

$$\frac{\Gamma, x : A \vdash B(x) \text{ Type}}{\Gamma \vdash \prod_{x:A} B(x) \text{ Type}}$$

- Introduction: $\frac{\Gamma, x : A \vdash t : B(x)}{\Gamma \vdash \lambda x. t : \prod_{x:A} B(x)}$

- Elimination: $\frac{\Gamma \vdash f : \prod_{x:A} B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)}$
 - Computation rule: $(\lambda x.t)(a) \equiv t[a/x]$
2. **Dependent sum** $\sum_{x:A} B(x)$: Formal existential quantification; pairs (a, b) with $b : B(a)$. Mathematical structures like topological spaces or subgroups are naturally encoded as $\sum$ -types.

$$\frac{\Gamma, x : A \vdash B(x) \text{ Type}}{\Gamma \vdash \sum_{x:A} B(x) \text{ Type}}$$

- Introduction rule: $\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B(a)}{\Gamma \vdash (a, b) : \sum_{x:A} B(x)}$
- Elimination: Recursor $\text{rec}_{\Sigma}(C, g, p)$; reduction $\text{rec}_{\Sigma}(C, g, (a, b)) \equiv g(a, b)$.

Distinction Between Type Theory and ZFC Syntax

1. Primitive relation: ZFC adopts $\in$ as a first-order predicate; type theory treats $a : A$ as an irreducible syntactic judgement.
2. Paradox prevention: ZFC adds regularity axioms as external restrictions; type theory's layered context syntax intrinsically blocks circular typing.
3. Axiomatic structure: ZFC uses disconnected axiom schemas; type theory relies on the uniform FIEC template for all type formers.

13.3 Curry-Howard isomorphism

Inductive type

Definition 11 Fix a valid context Γ . An expression A is a Γ -well-formed type if $\Gamma \vdash A \text{ Type}$ can be deduced through finite formation rules. There is no universal collection of all types, which syntactically blocks Russell-style paradoxes.

An **inductive type** is a well-formed type defined by the `Inductive` command in the Calculus of Inductive Constructions (CIC, the foundation of Coq). It supplies a finite list of constructors, which are the exclusive ways to build all elements of this type:

1. Constructors satisfy disjointness and injectivity: values from different constructors are never equal; equal constructor results imply equal input arguments.
2. Every inhabitant of the inductive type can only be built by finitely nested constructor applications.

It fully obeys the FIEC four-rule framework:

1. **Formation (F)**: Check the types parameters to guarantee well-formedness under Γ .
2. **Introduction (I)**: Constructors act as introduction rules to generate elements.
3. **Elimination (E)**: An associated recursor (induction principle) enables pattern matching and case analysis on values.
4. **Computation (C)**: Matching a recursor against a constructor term triggers definitional reduction, corresponding to Coq tactics `simpl`, `cbn`, `cbv`.

By the CurryHoward isomorphism, inductive types of sort `Prop` represent logical propositions: constructors correspond to inference rules, and type inhabitants correspond to formal proofs.

Example 12 Non-trivial inductive propositions under CurryHoward isomorphism (CIC)

1. **Less-than-or-equal predicate** `le`

```

Inductive le : nat -> nat -> Prop :=
| le_n   : forall n, le n n
| le_S   : forall n m, le n m -> le n (S m).

```

- Indexed inductive predicate; two constructors correspond to deduction rules of ordering.
- Proof term records derivation chains of inequalities; recursor supports induction over ordering proofs.

2. State reachability predicate reach

```

Inductive reach : State -> State -> Prop :=
| reach_refl  : forall s, reach s s
| reach_trans : forall s1 s2 s3, reach s1 s2 -> step s2 s3 -> reach s1 s3.

```

- Encodes transitive closure of state transition relations.
- Proof term stands for execution paths between states; recursor enables path induction.

3. Well-founded accessibility predicate Acc

```

Inductive Acc : A -> Prop :=
acc_intro : forall x, (forall y, R y x -> Acc y) -> Acc x.

```

- Core predicate for proving termination of recursive functions.
- Proof term forms well-founded induction tree; recursor derives well-founded induction principle.

Curry-Howard isomorphism: Proposition is a type, proof is a term

Theorem 13 (Curry-Howard isomorphism) Let $\mathcal{L}$ denote intuitionistic natural deduction logic, and $\mathcal{T}$ denote the Calculus of Inductive Constructions equipped with the FIEC rule system for inductive types. There exists a bijective structure-preserving mapping $\Phi : \mathcal{L} \leftrightarrow \mathcal{T}$ with the following mapping laws:

Logical Proposition $\longleftrightarrow$ Inductive type of sort Prop

Logical introduction inference rule $\longleftrightarrow$ Inductive type constructor (FIEC I-rule)

Formal proof of a proposition $\longleftrightarrow$ Well-typed inhabitant of the corresponding inductive type

Logical elimination rule case analysis $\longleftrightarrow$ Recursor rec_T (FIEC E-rule)

Proof normalization and cut elimination $\longleftrightarrow$ Definitional term reduction (FIEC C-rule)

Brief core correspondence: Propositions as Types, Proofs as Typed Terms, Proof Normalization as Term Computation.

核心洞见

证明的类型就是它所证明的命题。

Curry-Howard 同构

逻辑世界 $\longleftrightarrow$ 计算世界

- 命题 $\equiv$ 类型
- 证明 $\equiv$ 程序
- 证明检查 $\equiv$ 类型检查

深刻联系

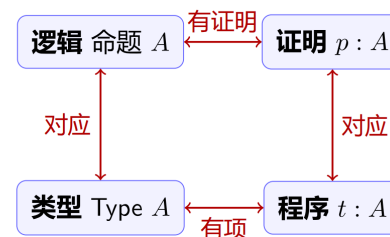


Figure 13.1: Global one-to-one structural correspondence of CurryHoward isomorphism

$$\Gamma \vdash p : A$$

环境 Γ	A 是对象类型	A 是命题
带有类型的名称列表	p 是类型 A 的项	p 是 A 的证明
$x : \text{nat}$ $h : x = 1$	$x : \text{nat} \vdash x : \text{nat}$ p 在环境 Γ 中格式正确	$h : x = 1 \vdash p : x \neq 0$ p 是证明的证据

Figure 13.2: Instance correspondence between inductive propositions and their proof terms

Example 14 Non-trivial inductive propositions instantiating the CurryHoward mapping**1. Order relation le on natural numbers**

```
Inductive le : nat -> nat -> Prop :=
| le_n : forall n, le n n
| le_S : forall n m, le n m -> le n (S m).
```

- Two constructors correspond to two inference rules for less-than-or-equal judgment;
- A proof term of `le 2 4` is a finite chain of constructor applications recording the derivation path;
- The recursor of `le` implements mathematical induction over inequality derivations.

2. State reachability transitive closure Given `State : Type` and one-step transition `step : State → State → Prop`:

```
Inductive reach : State -> State -> Prop :=
| reach_refl : forall s, reach s s
| reach_trans : forall s1 s2 s3, reach s1 s2 -> step s2 s3 -> reach s1 s3.
```

- Constructors encode reflexivity and transitive closure of transition relations;
- Each proof term of `reach  $s_0$   $s_t$` encodes a complete execution path from initial state to target state;
- Elimination via recursor enables inductive reasoning over all reachable paths.

3. Accessibility predicate Acc for well-founded recursion Let $R : A \rightarrow A \rightarrow \text{Prop}$ be a binary relation:

```
Inductive Acc : A -> Prop :=
acc_intro : forall x, (forall y, R y x -> Acc y) -> Acc x.
```

- Recursive single constructor formalizes the well-founded condition;
- Proof of `Acc  $x$` is an induction tree that guarantees termination of structural recursive functions;
- Its recursor provides the well-founded induction principle for termination proofs.

Remark Trivial propositions (`True`, `False`, $A \wedge B$, $A \vee B$) are special simple inductive types. Indexed inductive predicates such as `le`, `reach` and `Acc` demonstrate the full expressive power of the CurryHoward isomorphism, bridging inductive logical derivations and inductive type constructions uniformly under the FIEC rule framework.

Exercises

1. Construct a derivation tree proving $\Gamma \vdash A$ Type.
2. Explain why self-referential universe $\mathcal{U} : \mathcal{U}$ creates inconsistency, using well-formedness rules.

3. Formalize logical negation $\neg : \mathbb{B} \rightarrow \mathbb{B}$ with $\text{if}_{\mathbb{B}}$ and verify $\neg(\text{true}) \equiv \text{false}$.
4. Prove no closed term t satisfies $\emptyset \vdash t : \mathbf{0}$.
5. Write the FIEC rule set for product types and contrast it with ZFC pairing axiom.

13.4 Plan for next week

- Prove the results

Theorem 15 *There is an exact sequence:*

$$1 \longrightarrow \mathcal{P}_n \xrightarrow{\lambda} \mathcal{C}_n \xrightarrow{\pi} \text{Sp}(2n, \mathbb{Z}_2) \longrightarrow 1.$$

- Read the book QECC-2026.
- Learn about type theory and Lean4.

References

- [1] W. Huang. Lecture Notes: Formal Methods, Chapter 6.3 Coq Inductive Declarations. <https://faculty.ustc.edu.cn/huangwenchao>, 2024.
- [2] Lean Formalization Team. Theorem Proving in Lean 4 (Chinese Version), Chapter 7 Inductive Types. <https://www.leanprover.cn/tp-lean-zh/InductiveTypes/>.
- [3] Y. Bertot and P. Castéran. *Interactive Theorem Proving and Program Development: Coq'Art The Calculus of Inductive Constructions*, Springer, 2004.
- [4] W. A. Howard. The formulae-as-types notion of construction. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479-490. Academic Press, 1980.
- [5] C. Paulin-Mohring. Inductive definitions in the system Co: Rules and properties. In *Typed Lambda Calculi and Applications*, LNCS 664, pages 328-345. Springer, 1993.
- [6] H. P. Barendregt. *Lambda Calculus with Types*, Cambridge University Press, 2013.
- [7] P. Dybjer and A. Abel. Intuitionistic type theory. *Handbook of the History of Logic*, Volume 14, pages 261-340. Elsevier, 2019.